Cpp

Raccolta Completa: 7 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

Una **variabile** è una porzione di memoria RAM destinata a contenere un dato che può variare durante l'esecuzione del programma.

Possiamo immaginarla come una **scatola etichettata** con 4 caratteristiche inscindibili:

```mermaid
flowchart LR
    subgraph RAM ["Memoria RAM"]
        direction TB
        IND["Indirizzo: 0x7ffd5e"]
        NOME["Identificatore: eta"]
        TIPO["Tipo: int (4 byte)"]
        VAL["Valore: 18"]

        IND --- NOME --- TIPO --- VAL
    end
```

1. **Nome (Identificatore)**: l'etichetta usata per riferirsi alla cella (es. `eta`, `punteggio`).
2. **Tipo di Dato**: stabilisce quanti byte di RAM riservare e come interpretare i bit (`int`, `float`, `char`...).
3. **Valore**: il contenuto effettivo memorizzato nella cella.
4. **Indirizzo di Memoria**: la posizione fisica univoca in RAM (es. `0x7ffd5e8b41ac`), accessibile con l'operatore `&`.

> **NOTE**
>
> A differenza dei file (persistenti su SSD/hard disk), le variabili sono **volatili**: quando il programma termina o il computer si spegne, tutti i dati nelle variabili vengono distrutti.

---

1. Ciclo di Vita di una Variabile

Le tre fasi fondamentali per usare una variabile sono:

```
flowchart LR
    A["1. Dichiarazione\nint x;"] --> B["2. Inizializzazione\nx = 10;"] --> C["3. Ut
```

```
int eta;          // 1. Dichiarazione (valore iniziale indefinito / spazza
eta = 18;         // 2. Assegnazione (memorizza il valore)

int livello = 1;  // Inizializzazione contestuale (buona norma consigliata
```

---

2. I Tipi di Dato Primitivi

In C++ ogni variabile deve avere un tipo stabilito a priori (**tipizzazione statica e forte**).

Tipo	Significato	Dimensione	Intervallo / Valori possibili	Esempio
`int`	Numero intero con segno	4 byte (32 bit)	Da −2 × 109 a +2 × 109	`int monete = 350;`
`float`	Decimale singola precisione	4 byte (32 bit)	≈ 7 cifre decimali significative	`float peso = 68.5f;`
`double`	Decimale doppia precisione	8 byte (64 bit)	≈ 15 cifre decimali significative	`double pi = 3.14159265;`
`char`	Singolo carattere ASCII	1 byte (8 bit)	Tra apici singoli `' '` (codici 0-255)	`char voto = 'A';`
`bool`	Valore booleano di verità	1 byte	`true` (1) oppure `false` (0)	`bool vivo = true;`
`string`	Testo (classe STL da `<string>`)	Dinamica	Testo tra doppi apici `" "`	`string nome = "Luca";`

---

3. Modificatori, Costanti e Integer Overflow

Modificatori

- **`unsigned`**: elimina i negativi, raddoppiando il limite positivo (`unsigned int` va da 0 a oltre 4 miliardi).
- **`long long`**: estende gli interi a 8 byte (fino a ±9 × 1018).
- **`const`**: trasforma la variabile in una **costante di sola lettura**. Qualsiasi modifica successiva genererà un errore di compilazione.

```
const float TASSO_IVA = 0.22f;
// TASSO_IVA = 0.25f; // ERRORE! Non puoi modificare una costante!
```

> 🚫 **Il Fenomeno dell'Integer Overflow**
>
> Se aggiungi `1` al valore massimo consentito da un `int` a 32 bit (2.147.483.647), il bit di segno si ribalta e il numero “ricomincia” dal limite negativo:
>
> ```
> int max = 2147483647;
> max = max + 1;
> cout << max; // Stampa: -2147483648!
> ```

---

4. Input e Output (`cout`, `cin`, `getline()`)

Output con `cout` (Operatore `<<`)

```
int vite = 3;
cout << "Vite rimaste: " << vite << endl;
```

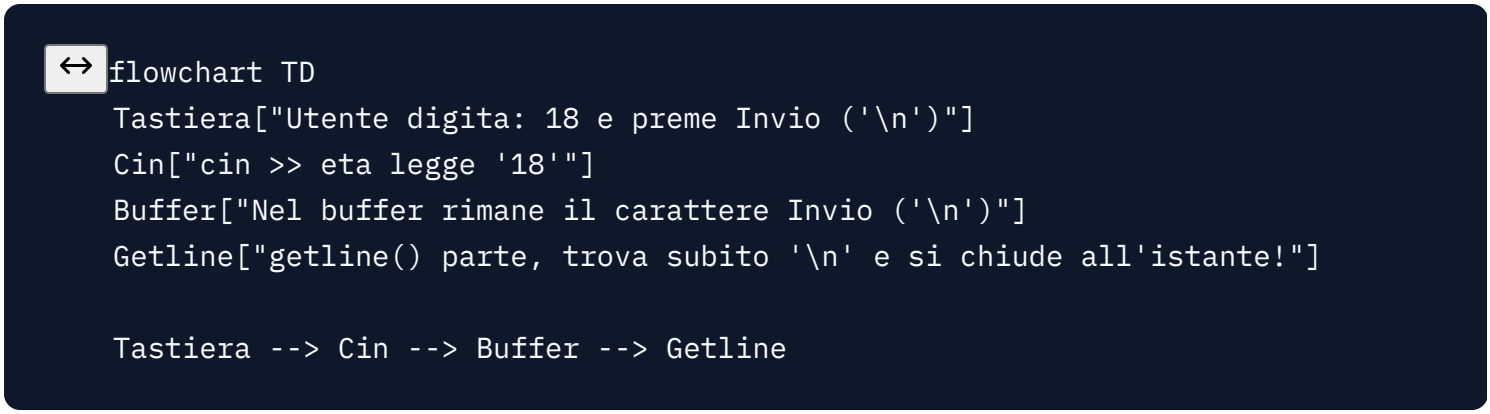
Input con `cin` (Operatore `>>`)

```
int eta;
cout << "Quanti anni hai? ";
cin >> eta;
```

> ⚠️ **Il Limite di `cin >>`**
>
> L'operatore `>>` **si ferma al primo spazio bianco**. Se inserisci `"Mario Rossi"`, `cin` leggerà solo `"Mario"` e lascerà `"Rossi"` nel canale di input!

Lettura con Spazi: `getline()` e la Trappola del Buffer

Per leggere frasi complete con spazi si usa `getline(cin, stringa)`.
Tuttavia, se usato subito dopo un `cin >>`, si verifica un errore comunissimo:



> 🎯 **La Soluzione: `cin.ignore()`**
>
> Inserisci sempre `cin.ignore()` per "ripulire" l'Invio pendente prima di chiamare `getline()`:

```
int eta;
string indirizzo;

cin >> eta;
cin.ignore(); // Svuota il buffer!
getline(cin, indirizzo); // Ora funziona regolarmente!
```

---

5. Scope e Variable Shadowing

Lo **scope** (ambito di visibilità) definisce dove una variabile è valida.

```
#include <iostream>
using namespace std;

int x = 100; // Variabile Globale

int main() {
    int x = 10; // Locale del main (nasconde quella globale: Shadowing!)

    {
        int x = 5; // Locale del blocco interno
        cout << "x blocco interno: " << x << endl; // 5
    } // Qui la x interna viene distrutta

    cout << "x locale main:    " << x << endl;   // 10
    cout << "x globale (::x):  " << ::x << endl; // 100
    return 0;
}
```

---

6. Typecasting con `static_cast`

> ❓ **Domanda d'Esame: Perché 5 / 2 dà 2 in C++?**

> Perché la divisione tra due interi genera **sempre un intero troncato**. Per avere decimali, almeno uno dei due operandi deve essere convertito con **`static_cast()`**:

```
int a = 5;
int b = 2;

// Errato (troncamento a 2):
float errato = a / b; // 2.0

// Corretto con static_cast:
float corretto = static_cast<float>(a) / b; // 2.5
```

1. Il Concetto di File e la Memoria di Massa

Un **file** è un'unità logica di memorizzazione allocata su memorie secondarie persistenti (SSD, hard disk, pendrive). A differenza delle variabili allocate nella memoria centrale RAM (che è volatile e si azzera allo spegnimento del computer), i file permettono la **conservazione duratura** dei dati.

Quando il nostro programma apre un file:

1. Il sistema operativo carica i blocchi dal supporto di massa a un **buffer in RAM**.
2. Il programma lavora sui dati in RAM.
3. Per rendere definitive le modifiche, i dati devono essere sincronizzati nuovamente su disco (*flush*), operazione che avviene automaticamente alla chiusura.

```
flowchart TD
    A["1. Apertura Flusso\n(ifstream / ofstream / fstream)"] --> B{"File aperto con
    B -->|"Sì"| C["2. Lavoro sui Dati\n(Lettura >> / getline o Scrittura << / write)
    B -->|"No"| ERR["Segnalazione Errore<br>(cerr << 'File non trovato')"]
    C --> D["3. Chiusura Risorsa\n(.close() o distruttore RAII)"]

    style A fill:#2b5b84,stroke:#306998,color:#fff
    style B fill:#ffe873,stroke:#ffd43b,color:#000
    style C fill:#d4edda,stroke:#28a745,color:#155724
    style D fill:#d1ecf1,stroke:#17a2b8,color:#0c5460
    style ERR fill:#f8d7da,stroke:#dc3545,color:#721c24
```

I file si distinguono principalmente in due categorie:

- **File di Testo (`.txt` , `.csv` , `.json`)**: sequenze di caratteri ASCII/UTF-8 organizzate in righe logiche delimitate da caratteri di a-capo (`\n`).
- **File Binari (`.bin` , `.dat` , `.exe`)**: sequenze grezze di byte che rispecchiano la rappresentazione nativa dei dati in memoria RAM.

---

2. L'Architettura Stream in C++ (`<fstream>`)

Nel C++ la gestione dei file è orientata agli oggetti e sfrutta il concetto di **Stream** (flusso continuo di dati).
La libreria standard mette a disposizione tre classi principali incluse nell'header `<fstream>`:

```mermaid
flowchart TD
    IOS["ios_base / basic_ios\n(Stato del flusso, flag e formattazione)"]
    IOS --> IS["istream\n(Operazioni di Input, cin)"]
    IOS --> OS["ostream\n(Operazioni di Output, cout)"]
    IS --> IFS["ifstream\n(Input File Stream: Lettura da file)"]
    OS --> OFS["ofstream\n(Output File Stream: Scrittura su file)"]
    IS --> FS["fstream\n(File Stream: Lettura E Scrittura)"]
    OS --> FS

    style IFS fill:#cce5ff,stroke:#007bff,color:#004085
    style OFS fill:#d4edda,stroke:#28a745,color:#155724
    style FS fill:#fff3cd,stroke:#ffc107,color:#856404
```

1. **`ifstream`** (*Input File Stream*): apre file in sola **lettura**.
2. **`ofstream`** (*Output File Stream*): crea o apre file in **scrittura**.
3. **`fstream`**: flusso bidirezionale in grado di eseguire sia **lettura che scrittura** sullo stesso file.

> 💡 **Il Paradigma RAII e la Chiusura Automatica**
>
> In C++ non è strettamente obbligatorio chiamare `file.close()` se l'oggetto stream è dichiarato localmente in una funzione o blocco. Grazie al paradigma **RAII** (*Resource Acquisition Is Initialization*), quando la variabile stream esce dal suo ambito di validità (`}`), il distruttore viene invocato automaticamente chiudendo il file e svuotando il buffer, anche in presenza di `return` anticipati o eccezioni!

---

3. Modalità di Apertura ed i Flag `ios::*`

Quando apriamo un file, possiamo specificare una o più modalità combinandole tramite l'operatore bitwise OR `|`:

fstream file("archivio.dat", ios :: in | ios :: out | ios :: binary);

Flag	Nome Completo	Descrizione e Comportamento
`ios::in`	Input	Apre il file per la **lettura** (impostato di default per `ifstream`).
`ios::out`	Output	Apre il file per la **scrittura** (impostato di default per `ofstream`). **Azzera il file** se esiste già!
`ios::app`	Append	**Aggiunta in coda**: posiziona il cursore alla fine. I vecchi dati non vengono toccati.
`ios::trunc`	Truncate	**Troncamento**: cancella completamente il contenuto preesistente all'apertura.
`ios::binary`	Binary	**Modalità binaria pura**: disattiva ogni traduzione di caratteri speciali di a-capo.
`ios::ate`	At The End	Apre il file e sposta immediatamente il cursore alla fine, ma consente di riposizionarsi altrove.

Perché `ios::binary` è Cruciale?

Nei sistemi operativi Windows, un a-capo nei file di testo è codificato con due byte: Carriage Return (`\r`, ASCII 13) e Line Feed (`\n`, ASCII 10). In modalità testo standard, il C++ converte automaticamente la sequenza `\r\n` in un singolo `\n` durante la lettura, e viceversa in scrittura. Nei **file binari** questa traduzione corrompe irreversibilmente immagini, numeri float o struct! Con `ios::binary`, il flusso tratta i dati come byte esatti senza alcuna alterazione.

```
// Esempio: Creare un file di testo in modalità append (senza sovrascrivere
ofstream fileLog("eventi.log", ios::out | ios::app);

// Esempio: Aprire un file binario sia in lettura che in scrittura
fstream fileDati("archivio.dat", ios::in | ios::out | ios::binary);
```

---

4. Controllo dello Stato del Flusso e Gestione Errori

Ogni oggetto stream mantiene una maschera interna di bit che descrive lo stato del canale di comunicazione:

```
flowchart LR
    FLUSSO["Stato Stream"]
    FLUSSO --> GOOD[".good() -> Flusso integro e funzionante"]
    FLUSSO --> FAIL[".fail() -> Errore logico di formato o dato non corrispondente"]
    FLUSSO --> BAD[".bad() -> Errore critico hardware o di sistema"]
    FLUSSO --> EOF[".eof() -> Raggiunta la fine del file (End Of File)"]

    style GOOD fill:#d4edda,stroke:#28a745,color:#155724
    style FAIL fill:#fff3cd,stroke:#ffc107,color:#856404
    style BAD fill:#f8d7da,stroke:#dc3545,color:#721c24
    style EOF fill:#cce5ff,stroke:#007bff,color:#004085
```

- `file.is_open()` : verifica se il file è stato effettivamente aperto dal sistema operativo.
- `file.fail()` : vero se si è verificato un errore logico di estrazione (es. si cercava di leggere un numero ma nel file c'era una parola).
- `file.clear()` : **ripristina i flag di stato** azzerando gli errori, operazione indispensabile per riutilizzare lo stream dopo aver toccato l'EOF.

> 🚫 **Errore da Principiante:** `while (!file.eof())`
>
> **NON usare mai** `while (!file.eof())` **come condizione di lettura!**

```
// CODICE SBAGLIATO CHE DUPLICA L'ULTIMA RIGA:
while (!file.eof()) {
    file >> dato;
    cout << dato << endl; // Verrà stampato due volte l'ultimo valore
}
```

Il flag `eofbit` diventa `true` **solo DOPO** che un'operazione di lettura ha già fallito tentando di leggere oltre la fine!

Pattern Corretto Idiomatico:

```
while (file >> dato) {
    cout << dato << endl; // Eseguito SOLO se la lettura ha avuto pie
}
```

---

5. Operazioni su File di Testo (`.txt` , `.csv`)

A. Scrittura Formattata con `ofstream` e `<<`

Scrivere su file usa la medesima sintassi di `cout` :

```cpp
#include <iostream>
#include <fstream>
#include <iomanip>

int main() {
    std::ofstream fout("catalogo.txt");
    if (!fout.is_open()) {
        std::cerr << "Errore creazione file!" << std::endl;
        return 1;
    }

    fout << std::fixed << std::setprecision(2);
    fout << "Mouse_Wireless" << " " << 25.99 << std::endl;
    fout << "Tastiera_RGB" << " " << 59.50 << std::endl;
    fout << "Monitor_4K" << " " << 320.00 << std::endl;

    fout.close();
    return 0;
}
```

B. Lettura Parola per Parola con `>>`

L'operatore `>>` salta automaticamente spazi bianchi, tabulazioni e caratteri di a-capo:

```cpp
std::ifstream fin("catalogo.txt");
std::string prodotto;
double prezzo;

while (fin >> prodotto >> prezzo) {
    std::cout << "Articolo: " << prodotto << " | Prezzo: €" << prezzo << st
}
fin.close();
```

C. Lettura di Intere Righe con `std::getline()`

Quando le stringhe contengono spazi (nomi, descrizioni, frasi intere), si usa `std::getline(flusso, stringa, [delimitatore])`:

```cpp
#include <iostream>
#include <fstream>
#include <string>

int main() {
    std::ifstream file("rubrica.txt");
    std::string riga;

    // Legge fino al carattere '\n'
    while (std::getline(file, riga)) {
        std::cout << "Riga letta: " << riga << std::endl;
    }
    return 0;
}
```

Parsing di File CSV con Delimitatore Personalizzato

`getline` accetta un terzo parametro facoltativo: il **delimitatore** (di default `'\n'`). Possiamo usarlo per separare campi CSV delimitati da virgole o punti e virgola:

```cpp
std::ifstream fileCSV("studenti.csv");
std::string matricola, nome, media;

// Legge campi separati da virgola
while (std::getline(fileCSV, matricola, ',') &&
       std::getline(fileCSV, nome, ',') &&
       std::getline(fileCSV, media)) { // L'ultimo campo termina con \n!
    std::cout << "Matr: " << matricola << " | Nome: " << nome << " | Media
}
```

> ⚠️ **La Trappola del Buffer: `>>` seguito da `getline()`**
>
> Se alterni una lettura con `file >> numero` e una successiva con `std::getline(file, stringa)`, `getline()` leggerà immediatamente una **stringa vuota**! Questo accade perché

> `>>` estrae il numero ma lascia il carattere `\n` in sospeso nel buffer.
> **Soluzione**: inserisci sempre `file.ignore()` prima del `getline()` !

---

6. Operazioni su File Binari (`ios::binary`)

Nei file binari non si convertono i numeri in caratteri: si memorizza l'esatta rappresentazione a byte della memoria centrale RAM.

I metodi fondamentali sono:

- **`flusso.write(const char* buffer, streamsize num_byte)`**
- **`flusso.read(char* buffer, streamsize num_byte)`**

Entrambi i metodi richiedono che il puntatore alla variabile venga convertito in un puntatore a caratteri grezzi (`char*`) tramite l'operatore di cast del C++ moderno: **`reinterpret_cast`** .

```
flowchart LR
    RAM["Struttura in RAM\nstruct Player {\n  int id;\n  float hp;\n};"]
    RAM -->|reinterpret_cast<const char*>&p| WRITE["file.write(..., sizeof(Player))"]
    WRITE --> DISCO["File Binario su Disco\n[01 00 00 00  00 00 C8 42]\n(Byte grezzi
    DISCO -->|reinterpret_cast<char*>&p| READ["file.read(..., sizeof(Player))"]
    READ --> RAM2["Struttura Ricaricata in RAM"]

    style RAM fill:#cce5ff,stroke:#007bff,color:#004085
    style DISCO fill:#ffe873,stroke:#ffd43b,color:#000
    style RAM2 fill:#d4edda,stroke:#28a745,color:#155724
```

A. Scrittura Binaria di una `struct`

```cpp
#include <iostream>
#include <fstream>

struct Giocatore {
    int id;
    char username[24]; // Dimensione FISSA per allocazione contigua
    int livello;
    float punteggio;
};

int main() {
    Giocatore g1 = {1001, "CyberKnight", 15, 8450.5f};

    std::ofstream fout("salvataggio.dat", std::ios::out | std::ios::binary
    if (!fout.is_open()) return 1;

    // Scrive l'intero blocco di byte della struct in un unico colpo
    fout.write(reinterpret_cast<const char*>(&g1), sizeof(Giocatore));
    fout.close();

    std::cout << "Salvataggio completato (" << sizeof(Giocatore) << " byte
    return 0;
}
```

B. Lettura Binaria con Verifica dei Byte Effettivi

Il metodo `file.gcount()` restituisce il numero esatto di byte effettivamente estratti dall'ultima chiamata `.read()`:

```
Giocatore caricato;
std::ifstream fin("salvataggio.dat", std::ios::in | std::ios::binary);

if (fin.is_open()) {
    fin.read(reinterpret_cast<char*>(&caricato), sizeof(Giocatore));

    if (fin.gcount() == sizeof(Giocatore)) {
        std::cout << "Giocatore: " << caricato.username
                  << " | Livello: " << caricato.livello
                  << " | Punti: " << caricato.punteggio << std::endl;
    }
    fin.close();
}
```

> 🚫 **Trappola Mortale: MAI usare `std::string` o Puntatori grezzi dentro struct per file binari!**
>
> Una classe dinamica come `std::string` memorizza i caratteri del testo nello **Heap** e possiede al suo interno solo un puntatore all'indirizzo RAM. Se scrivi una `std::string` con `file.write(&str, sizeof(str))`, salverai sul disco l'**indirizzo di memoria virtuale**, non il testo!
> Quando riaprirai il file in una successiva esecuzione, quell'indirizzo conterrà dati casuali o non validi, provocando un disastroso crash per **Segmentation Fault**.
> **Regola d'oro per file binari**: usa array di caratteri a dimensione fissa (`char nome[32]`) o strutture composte solo da tipi primitivi (dette *Trivially Copyable*).

---

7. Accesso Diretto (Random Access): `seekg`, `seekp`, `tellg`, `tellp`

Nei file binari e nei database non serve rileggere tutto da capo per consultare o modificare un record: possiamo saltare istantaneamente all'offset di byte desiderato.

I due flussi hanno puntatori indipendenti:

- `g` **(get)**: cursore del flusso di **lettura** (`seekg` e `tellg`).
- `p` **(put)**: cursore del flusso di **scrittura** (`seekp` e `tellp`).

Sintassi di Posizionamento:

file.seekg(offset, punto_riferimento); file.seekp(offset, punto_riferimento);

I punti di riferimento sono definiti nell'oggetto `std::ios` :

1. `std::ios::beg` : Inizio del file (offset positivo verso destra).
2. `std::ios::cur` : Posizione attuale del cursore.
3. `std::ios::end` : Fine del file (offset tipicamente negativo a ritroso).

Trucco: Calcolare la Dimensione di Qualsiasi File in C++

```
std::ifstream file("video.mp4", std::ios::binary | std::ios::ate);
if (file.is_open()) {
    // Poiché abbiamo usato ios::ate, il cursore si trova già alla fine!
    std::streamsize grandezza = file.tellg();
    std::cout << "Peso del file: " << grandezza << " byte ("
              << grandezza / (1024.0 * 1024.0) << " MB)" << std::endl;
    file.close();
}
```

Modifica di un Record Specifico in Mezzo al File

```
std::fstream fs("database.dat", std::ios::in | std::ios::out | std::ios::b

int indice_record = 3; // Vogliamo modificare il quarto record (indice 3)
long offset = indice_record * sizeof(Giocatore);

// 1. Spostiamo il puntatore di lettura sul record e lo leggiamo
fs.seekg(offset, std::ios::beg);
Giocatore g;
fs.read(reinterpret_cast<char*>(&g), sizeof(Giocatore));

// 2. Modifichiamo il dato in memoria
g.livello += 1;

// 3. Spostiamo il puntatore di SCRITTURA sullo stesso offset per sovrascr
fs.seekp(offset, std::ios::beg);
fs.write(reinterpret_cast<const char*>(&g), sizeof(Giocatore));

fs.close();
```

---

8. Domande d'Esame e Concetti Chiave

> ❓ **Domande di Verifica**
>
> 1. *Cosa accade se ometti il flag `ios::binary` quando salvi una struct su file in Windows?*
>   **Risposta**: Qualsiasi byte all'interno dei dati binari della struct avente valore 10 (Line Feed `\n`) verrà trasformato automaticamente nella coppia di byte 13, 10 (`\r\n`), alterando la dimensione dei dati e corrompendo la struttura al caricamento successivo.
> 2. *Qual è la differenza pratica tra `file.seekg(10, ios::beg)` e `file.seekg(10, ios::cur)`?*
>   **Risposta**: `ios::beg` posiziona il cursore esattamente al decimo byte a partire dall'inizio del file; `ios::cur` sposta il cursore di 10 byte in avanti a partire dalla posizione in cui si trova in quel momento.
> 3. *Perché `getline(cin, s)` fallisce se preceduta da un'operazione `cin >> x`? Come si risolve?*

Risposta: L'operatore `>>` consuma il valore numerico ma lascia il carattere `\n` generato dal tasto Invio all'interno del buffer di input. La successiva `getline()` incontra subito quel `\n` e termina immediatamente restituendo una stringa vuota. Si risolve invocando `cin.ignore()` per svuotare il carattere residuo.

---

9. Programmi Esecutivi Completi in C++

🧪 Programma 1: Gestione Anagrafica Studenti con File CSV e `getline()`

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <sstream>

struct Studente {
    std::string cognome;
    std::string nome;
    float media;
};

const std::string NOME_FILE = "studenti.csv";

void inserisci_studente(const Studente& s) {
    std::ofstream fout(NOME_FILE, std::ios::out | std::ios::app);
    if (fout.is_open()) {
        fout << s.cognome << "," << s.nome << "," << s.media << "\n";
        fout.close();
        std::cout << "✅ Salvato: " << s.cognome << " " << s.nome <<
    }
}

void stampa_registro() {
    std::ifstream fin(NOME_FILE);
    if (!fin.is_open()) {
        std::cout << "⚠️ Registro vuoto o non trovato." << std::endl;
        return;
    }

    std::string riga;
    std::cout << "\n--- REGISTRO SCOLASTICO CSV ---" << std::endl;
    std::cout << "Cognome             Nome                Media" << st
    std::cout << "-----------------------------------------------" <<

    while (std::getline(fin, riga)) {
        if (riga.empty()) continue;

        std::stringstream ss(riga);
        std::string cognome, nome, str_media;
```

```
        if (std::getline(ss, cognome, ',') &&
            std::getline(ss, nome, ',') &&
            std::getline(ss, str_media)) {

            float media = std::stof(str_media);
            std::cout << cognome << "\t\t" << nome << "\t\t" << media
        }
    }
    fin.close();
}

int main() {
    inserisci_studente({"Casetti", "Alessandro", 8.9f});
    inserisci_studente({"Verdi", "Giuseppe", 7.5f});
    inserisci_studente({"Bianchi", "Laura", 9.4f});

    stampa_registro();
    return 0;
}
```

🧪 Programma 2: Database Binario di Record di Gioco con Accesso Diretto (Seek/Tell)

```
#include <iostream>
#include <fstream>
#include <cstring>

struct RecordAccount {
    int id;
    char nickname[24];
    int monete;
    bool attivo;
};

const char* DB_FILE = "accounts.bin";

void inizializza_db() {
    std::ofstream fout(DB_FILE, std::ios::out | std::ios::binary | st
    RecordAccount accts[3] = {
        {1, "GamerPro99", 500, true},
        {2, "ShadowBlade", 1200, true},
        {3, "PixelHero", 350, false}
    };

    fout.write(reinterpret_cast<const char*>(accts), sizeof(accts));
    fout.close();
    std::cout << "Database binario creato con 3 account iniziali." <<
}

void aggiungi_monete(int indice_account, int bonus) {
    // Apertura bidirezionale binaria
    std::fstream fs(DB_FILE, std::ios::in | std::ios::out | std::ios:
    if (!fs.is_open()) {
        std::cerr << "Errore apertura database!" << std::endl;
        return;
    }

    long pos = indice_account * sizeof(RecordAccount);

    // 1. Posizionamento in lettura
    fs.seekg(pos, std::ios::beg);
    RecordAccount acc;
    fs.read(reinterpret_cast<char*>(&acc), sizeof(RecordAccount));
```

```
    if (fs.gcount() == sizeof(RecordAccount)) {
        std::cout << "Monete attuali per " << acc.nickname << ": " <<
        acc.monete += bonus;

        // 2. Posizionamento nello stesso offset in scrittura
        fs.seekp(pos, std::ios::beg);
        fs.write(reinterpret_cast<const char*>(&acc), sizeof(RecordAc
        std::cout << "✅ Accreditate " << bonus << " monete. Nuovo sa
    }
    fs.close();
}

void mostra_tutti() {
    std::ifstream fin(DB_FILE, std::ios::in | std::ios::binary);
    RecordAccount temp;
    std::cout << "\n--- STATO FINALE DATABASE BINARIO ---" << std::en
    while (fin.read(reinterpret_cast<char*>(&temp), sizeof(RecordAcco
        std::cout << "ID: " << temp.id
                  << " | Nick: " << temp.nickname
                  << " | Monete: " << temp.monete
                  << " | Attivo: " << (temp.attivo ? "SÌ" : "NO") << 
    }
    fin.close();
}

int main() {
    inizializza_db();
    // Aggiungiamo 300 monete al secondo account (indice 1: ShadowBla
    aggiungi_monete(1, 300);
    mostra_tutti();
    return 0;
}
```

In Sintesi (A Colpo d'Occhio)

- **Array**: sequenza di celle di memoria RAM **contigue** contenenti elementi dello **stesso tipo**.
- **Indici**: da **`0`** a **`N - 1`** (l'indice rappresenta l'offset dall'indirizzo base).
- **Due Dimensioni**: **Fisica** (capacità massima dichiarata) vs **Logica** (dati effettivamente inseriti).
- **Pericolo Mortale**: **Out-of-Bounds** (il C++ non controlla i limiti; accedere oltre causa crash o corruzione dati).
- **Dinamicità**: per array a dimensione variabile si usa **`std::vector`** (`.push_back()` , `.size()`).

Un **array** (o **vettore**) è una struttura dati statica e omogenea che memorizza una sequenza di elementi dello stesso tipo in posizioni di memoria RAM strettamente contigue.

```
flowchart LR
    subgraph RAM ["Mappa di Memoria Contigua di un int arr[4]"]
        direction LR
        C0["Indice [0]<br>Valore: 10<br>Indirizzo: 0x100"]
        C1["Indice [1]<br>Valore: 20<br>Indirizzo: 0x104"]
        C2["Indice [2]<br>Valore: 30<br>Indirizzo: 0x108"]
        C3["Indice [3]<br>Valore: 40<br>Indirizzo: 0x10C"]

        C0 --- C1 --- C2 --- C3
    end
```

Domanda d'Esame: Perché gli indici partono da 0?

L'indice rappresenta l'**offset (scostamento)** rispetto alla prima cella di memoria:
Indirizzo(arr[i]) = IndirizzoBase + (i × dimensione_tipo)

Per il primo elemento lo scostamento è zero (0 × 4 byte = 0), quindi si trova esattamente all'indirizzo base!

1. Dimensione Fisica vs Dimensione Logica

Negli esercizi reali quasi mai un array statico viene riempito al 100% all'avvio:

```
flowchart LR
    subgraph FISICA ["Dimensione Fisica (MAX = 100 celle allocate)"]
        subgraph LOGICA ["Dimensione Logica (n = 3 celle usate)"]
            A["arr[0]: 15"] --- B["arr[1]: 28"] --- C["arr[2]: 42"]
        end
        D["arr[3]: ?"] --- E["..."] --- F["arr[99]: ?"]
        C --- D
    end
```

- **Dimensione Fisica (Capacità Massima)**: la memoria riservata in compilazione (es. `const int MAX = 100; int arr[MAX];`).
- **Dimensione Logica (Elementi Utili)**: il numero reale di dati inseriti dall'utente (`int n; cin >> n;`).

```
#include <iostream>
using namespace std;

int main() {
    const int MAX = 50; // Dimensione Fisica
    int voti[MAX];
    int n;              // Dimensione Logica

    do {
        cout << "Quanti voti vuoi caricare (max " << MAX << ")? ";
        cin >> n;
    } while (n < 1 || n > MAX);

    for (int i = 0; i < n; i++) {
        cout << "Voto #" << (i + 1) << ": ";
        cin >> voti[i];
    }
    return 0;
}
```

2. Dichiarazione e Inizializzazione

```
// 1. Non inizializzato (contiene dati spazzatura casuali!)
int dati[5];

// 2. Inizializzazione completa
int numeri[5] = {10, 20, 30, 40, 50};

// 3. Tutti gli elementi azzerati a 0
int zeri[5] = {0};

// 4. Parziale (i mancanti diventano automaticamente 0)
int primiDue[5] = {3, 7}; // Diventa: 3, 7, 0, 0, 0
```

💡 Calcolo Automatico della Dimensione con `sizeof`

```
int arr[] = {12, 45, 78, 23, 56};
int dimensione = sizeof(arr) / sizeof(arr[0]); // (5 * 4 byte) / 4 by
```

3. Il Pericolo “Out-of-Bounds”

🚫 Errore Gravissimo: Accesso Fuori Limiti

In un array `int arr[5]`, gli indici validi vanno da `0` a `4`.
Il C++ **non controlla i limiti a runtime** per massimizzare le prestazioni.
Se tenti di accedere a `arr[5]` o `arr[10]`, leggerai o sovrascriverai memoria casuale, causando crash improvvisi (*Segmentation Fault*) o corruzione silenziosa dei dati!

> 🖼️ **Placeholder Immagine: Rappresentazione dell'errore di memoria Out-of-Bounds**
>
> *Suggerimento per Obsidian: inserisci qui uno screenshot esplicativo dell'accesso a memoria non autorizzata oltre la fine dell'array.*
> `![[Pasted image out_of_bounds.png|550]]`

---

4. Scorrimento con Cicli `for`

Metodo 1: Ciclo ad Indice Classico

```
const int N = 4;
int numeri[N] = {5, 10, 15, 20};

for (int i = 0; i < N; i++) {
    cout << "arr[" << i << "] = " << numeri[i] << endl;
}
```

Metodo 2: Range-based for (C++11)

```
// Lettura per copia
for (int x : numeri) { cout << x << " "; }

// Modifica tramite riferimento (&)
for (int &x : numeri) { x *= 2; } // Raddoppia direttamente gli elementi
```

---

5. Algoritmi Fondamentali su Array

```cpp
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int arr[N] = {18, 4, 32, 9, 21};

    // 1. Somma e Media
    int somma = 0;
    for (int i = 0; i < N; i++) somma += arr[i];
    float media = static_cast<float>(somma) / N;

    // 2. Massimo e Minimo con Indice
    int max = arr[0], min = arr[0];
    int idxMax = 0, idxMin = 0;

    for (int i = 1; i < N; i++) {
        if (arr[i] > max) { max = arr[i]; idxMax = i; }
        if (arr[i] < min) { min = arr[i]; idxMin = i; }
    }

    cout << "Media: " << media << " | Max: " << max << " all'indice " << i
    return 0;
}
```

---

6. Array Bidimensionali (Le Matrici)

Una matrice è una tabella a griglia bidimensionale organizzata per **righe** e **colonne**:

```
flowchart TD
    subgraph MATRICE ["Matrice 2x3: int m[2][3]"]
        R0["Riga 0: [ m[0][0] ]  [ m[0][1] ]  [ m[0][2] ]"]
        R1["Riga 1: [ m[1][0] ]  [ m[1][1] ]  [ m[1][2] ]"]
        R0 --- R1
    end
```

```
int matrice[2][3] = {
    {10, 20, 30}, // Riga 0
    {40, 50, 60}  // Riga 1
};

// Scorrimento a griglia con cicli for annidati
for (int r = 0; r < 2; r++) {
    for (int c = 0; c < 3; c++) {
        cout << matrice[r][c] << "\t";
    }
    cout << endl;
}
```

---

7. Array Statici vs Array Dinamici (`std::vector`)

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v; // Array dinamico inizialmente vuoto

    v.push_back(10); // Inserisce in fondo
    v.push_back(20);

    cout << "Dimensione: " << v.size() << endl; // 2
    cout << "Accesso sicuro: " << v.at(0) << endl; // 10
    v.pop_back(); // Rimuove l'ultimo elemento
    return 0;
}
```

Proprietà	Array Statico (`int arr[N]`)	Vettore STL (`vector<int> v`)
Dimensione	Rigida in compilazione	**Dinamica ed espandibile**
Allocazione	Nello Stack	Nello Heap gestita in automatico
Aggiunta elementi	Impossibile oltre N	Metodo **`.push_back(valore)`**
Controllo limiti	Assente	Metodo sicuro **`.at(indice)`**

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Modularità**: spezzare un grande programma in piccoli sottoprogrammi indipendenti (Principio **DRY - Don't Repeat Yourself**).
> - **Funzione vs Procedura**: la **Funzione** calcola e restituisce un valore (`return`); la **Procedura** compie un'azione e non restituisce nulla (`void`).
> - **Parametri Formali vs Attuali**: i **formali** sono le variabili segnaposto nella firma; gli **attuali** sono i valori reali passati alla chiamata.
> - **Passaggio Parametri**: per **valore** (copia protetta), per **riferimento `&`** (modifica l'originale), per **riferimento costante `const &`** (massima velocità in sola lettura).

Nei programmi reali scrivere tutto il codice all'interno del `main()` genera software disordinato, difficile da comprendere e quasi impossibile da testare o correggere.

La soluzione architetturale è la **modularizzazione**: dividere un problema complesso in sotto-problemi più piccoli, implementati tramite **funzioni** e **procedure**.

```mermaid
flowchart TD
    M["Funzione Principale: main()"]
    F1["leggiDati()"]
    F2["calcolaMedia()"]
    F3["stampaReport()"]

    M -->|"1. Input dati"| F1
    M -->|"2. Elaborazione"| F2
    M -->|"3. Output risultati"| F3
```

---

1. I Grandi Vantaggi della Modularizzazione

- **Riusabilità (Principio DRY)**: scrivi una routine complessa **una sola volta** e la richiami da qualsiasi parte del programma.
- **Leggibilità**: il `main()` diventa un indice compatto e chiaro delle operazioni svolte ad alto livello.

- **Isolamento dei bug**: se si verifica un errore, puoi testare la singola funzione in modo indipendente senza dover riesaminare l'intero programma.
- **Lavoro in team**: programmatori diversi possono sviluppare e testare funzioni diverse in parallelo.

---

2. Funzioni vs Procedure

In C++ la sintassi di base è identica, ma concettualmente si distinguono due ruoli:

Proprietà	Funzione	Procedura
Tipo di Ritorno	Un tipo di dato concreto (`int` , `float` , `string` , `bool` ...)	`void` (nessun valore restituito)
Scopo	Riceve input, **calcola un risultato** e lo restituisce al chiamante	Compie un'**azione** (stampa a schermo, modifica file, ecc.)
Istruzione `return`	**Obbligatoria**: `return valore;`	Opzionale (si usa solo `return;` per uscire in anticipo)
Uso nel chiamante	Assegnata a una variabile o stampata: `x = radice(16);`	Invocata come comando isolato: `pulisciSchermo();`

> 💡 **Come riconoscerle subito**
>
> Se il sottoprogramma serve a **calcolare qualcosa** che ti serve nel codice successivo, usa una **Funzione**.
> Se serve solo a **mostrare qualcosa** o **eseguire una sequenza di comandi**, usa una **Procedura (`void`)**.

---

3. Parametri Formali vs Parametri Attuali (Argomenti)

> ❓ **Domanda Tipica da Interrogazione / Esame**
>
> **Qual è l'esatta differenza tra un parametro formale e un parametro attuale?**
>
> - Il **Parametro Formale** è la variabile dichiarata nella firma della funzione (il "segnaposto").
> - Il **Parametro Attuale** è il valore o la variabile reale fornita durante l'invocazione della funzione.

```
// Definizione della funzione:
int moltiplica(int a, int b)   // <-- 'a' e 'b' sono PARAMETRI FORMALI
{
    return a * b;
}

int main()
{
    int x = 4;
    int risultato = moltiplica(x, 5);  // <-- 'x' e '5' sono PARAMETRI ATTU
}
```

```
flowchart LR
    subgraph CHIAMANTE ["Contesto Chiamante: main()"]
        PA["Parametri Attuali: (x = 4, 5)"]
    end
    subgraph FUNZIONE ["Contesto Funzione: moltiplica(a, b)"]
        PF["Parametri Formali: (int a, int b)"]
    end

    PA -->|"Binding / Passaggio dei dati"| PF
```

Tabella Comparativa di Dettaglio

Aspetto	Parametri Formali	Parametri Attuali (Argomenti)
Dove si trovano?	Nell'**intestazione (firma)** della definizione o prototipo.	Nella **chiamata** alla funzione (es. dentro il `main`).
Cosa sono?	Variabili "segnaposto" con tipo e nome (`int a, int b`).	I valori, variabili o espressioni reali passati (`x`, `5`, `y + 2`).
Durata di vita	Esistono solo finché la funzione è in esecuzione nello Stack.	Esistono nel contesto di chi esegue la chiamata.

---

4. Anatomia di una Funzione

```
tipo_ritorno nomeFunzione(tipo1 param1, tipo2 param2) {
    // Corpo della funzione: logica e calcoli
    return espressione; // Restituisce il valore al chiamante
}
```

1. **Tipo di Ritorno**: specifica il tipo del dato prodotto (`int`, `double`, `bool`, `void` per procedure).
2. **Identificatore (Nome)**: segue le stesse regole dei nomi di variabile (es. `calcolaIva`, `massimoTraDue`).
3. **Parametri Formali**: elenco tra parentesi tonde dei dati in ingresso con il relativo tipo.
4. **Corpo**: racchiuso tra parentesi graffe `{ }`, contiene le istruzioni da eseguire.

---

5. I Prototipi di Funzione (Forward Declaration)

Il compilatore C++ legge il codice dall'alto verso il basso. Se incontra una chiamata prima che la funzione sia definita, solleva un errore di *"funzione non dichiarata"*.

Per organizzare il file in modo pulito ed elegante (mettendo il `main()` in alto e le funzioni sotto), si usano i **prototipi**:

```cpp
#include <iostream>
using namespace std;

// 1. PROTOTIPI (annunciano al compilatore l'esistenza della funzione)
int quadrato(int n);
void stampaSeparatore();

// 2. MAIN (subito visibile in cima al sorgente)
int main() {
    stampaSeparatore();
    cout << "Il quadrato di 6 e': " << quadrato(6) << endl;
    stampaSeparatore();
    return 0;
}

// 3. DEFINIZIONE DELLE FUNZIONI
int quadrato(int n) {
    return n * n;
}

void stampaSeparatore() {
    cout << "----------------------------------" << endl;
}
```

---

6. Modalità di Passaggio dei Parametri

Questa è una delle distinzioni fondamentali del C++.

```
flowchart TD
    subgraph MODALITA ["Le 3 Modalità di Passaggio a Confronto"]
        V["1. Per Valore (Copia)<br>• Crea una copia locale<br>• Nessun effetto sul
        R["2. Per Riferimento (&)<br>• Crea un alias diretto<br>• Modifica l'origina
        CR["3. Per Riferimento Costante (const &)<br>• Zero copie in RAM (massima ve
    end
```

A. Passaggio per Valore (Copia Protetta)

- **Come funziona**: il parametro formale riceve una **copia indipendente** del valore.
- **Effetto**: se modifichi il parametro all'interno della funzione, la variabile originale nel `main()` **non cambia**.

```
void raddoppiaFalso(int x) {
    x = x * 2; // Modifica solo la copia locale nello Stack frame
}

int main() {
    int valore = 10;
    raddoppiaFalso(valore);
    cout << valore << endl; // Stampa ancora 10!
}
```

B. Passaggio per Riferimento (`&`)

- **Come funziona**: inserendo il simbolo `&` dopo il tipo, il parametro formale non crea una nuova variabile ma diventa un **alias diretto** della variabile originale.
- **Effetto**: qualunque modifica fatta all'interno della funzione **modifica istantaneamente la variabile originale nel chiamante** !

> 🎯 **Caso d'Uso Tipico: La funzione Swap (Scambio)**
>
> ```
> void scambia(int &a, int &b) {
>     int temp = a;
>     a = b;
>     b = temp;
> }
> ```
>
> Poiché `a` e `b` hanno l'operatore `&` , la modifica scambia fisicamente le variabili passate nel `main()` .

C. Passaggio per Riferimento Costante (`const &`)

Quando passiamo oggetti pesanti (come stringhe lunghe o grandi strutture), il passaggio per valore è lento perché obbliga a copiare migliaia di byte.
Usando `const Tipo &` otteniamo:

1. 🚀 **Velocità massima**: zero copie in memoria.
2. 🛡️ **Sicurezza totale**: il prefisso `const` impedisce modifiche accidentali all'originale.

```
void analizzaTesto(const string &testo) {
    cout << "Lunghezza: " << testo.length() << endl;
    // testo = "modifica"; // ERRORE in compilazione: const impedisce modi
}
```

---

7. Il Call Stack e lo Stack Frame

Cosa succede nella memoria RAM quando invochi una funzione?

1. Nel momento della chiamata, il programma alloca nello **Stack** una nuova porzione di memoria chiamata **Stack Frame** (o record di attivazione).
2. Lo Stack Frame contiene:
   - I parametri formali ricevuti;
   - Le variabili locali create nella funzione;
   - L'indirizzo di ritorno (dove tornare nel `main` una volta finito).
3. Quando la funzione incontra `return` o la parentesi finale `}`, il suo Stack Frame viene **distrutto all'istante** e la memoria viene liberata.

```
↔sequenceDiagram
    participant Main as main() Stack Frame
    participant Func as calcolaArea() Stack Frame

    Main->>Func: Invocazione con argomenti (base=5, altezza=4)
    Note over Func: Alloca variabili locali nello Stack
    Func-->>Main: Restituisce valore (20) e distrugge lo Stack Frame
    Note over Main: Riprende l'esecuzione con il risultato
```

>

> 🖼️ **Placeholder Immagine: Rappresentazione dello Stack Frame in memoria RAM**
>
> *Suggerimento per Obsidian: inserisci qui uno schema visivo della memoria Stack con i record di attivazione impilati l'uno sull'altro.*
> `![[Pasted image stack_frame.png|550]]`

---

8. Overloading delle Funzioni (Sovraccarico)

In C++ è consentito definire **più funzioni con lo stesso identico nome**, purché abbiano una lista parametri diversa per numero o tipo (*firma univoca*). Il compilatore capirà automaticamente quale chiamare:

```
int somma(int a, int b) { return a + b; }            // Interi
double somma(double a, double b) { return a + b; }  // Decimali
int somma(int a, int b, int c) { return a + b + c; } // Tre parametri
```

---

9. La Ricorsione

Una funzione si definisce **ricorsiva** quando chiama se stessa per risolvere una porzione ridotta del problema originario.

```
flowchart TD
    F5["fattoriale(5) = 5 * fattoriale(4)"]
    F4["fattoriale(4) = 4 * fattoriale(3)"]
    F3["fattoriale(3) = 3 * fattoriale(2)"]
    F2["fattoriale(2) = 2 * fattoriale(1)"]
    F1["fattoriale(1) = 1 (Caso Base)"]

    F5 --> F4 --> F3 --> F2 --> F1
    F1 -.->|Risoluzione e ritorno| F2
    F2 -.-> F3
    F3 -.-> F4
    F4 -.-> F5
```

🚫 Errore Critico: Manca il Caso Base = Stack Overflow

Se una funzione ricorsiva non ha una corretta condizione di arresto (caso base), continuerà a chiamare se stessa all'infinito saturando la memoria dello Stack fino a provocare il crash immediato del programma (*Stack Overflow*).

```
long long fattoriale(int n) {
    if (n <= 1) return 1;           // 1. Caso Base (condizione di arresto)
    return n * fattoriale(n - 1);   // 2. Passo Ricorsivo
}
```

🧪 Programma Completo di Test Eseguibile (Clicca per espandere)

```cpp
#include <iostream>
using namespace std;

void scambia(int &a, int &b);
long long fattoriale(int n);

int main() {
    int x = 10, y = 20;
    cout << "Prima dello swap: x=" << x << ", y=" << y << endl;
    scambia(x, y);
    cout << "Dopo lo swap:    x=" << x << ", y=" << y << endl;

    int num = 5;
    cout << "Fattoriale di " << num << ": " << fattoriale(num) << endl
    return 0;
}

void scambia(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

long long fattoriale(int n) {
    if (n <= 1) return 1;
    return n * fattoriale(n - 1);
}
```

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Classe vs Oggetto**: la `Classe` è il progetto (*blueprint*); l'`Oggetto` è l'istanza concreta creata in RAM.
> - **L'Unica Differenza C++**: nelle `struct` i membri sono `public` di default; nelle `class` sono `private`.
> - **Incapsulamento**: variabili protette in `private`, accessibili dall'esterno solo tramite metodi `public` (getter e setter).
> - **Costruttore**: metodo speciale con lo stesso nome della classe per inizializzare l'oggetto alla nascita.
> - **Operatore Freccia (`->`)**: scorciatoia elegante per accedere ai membri tramite un puntatore (`ptr->metodo()`).

---

1. Classe vs Oggetto (Il Progetto e la Casa)

- **Classe (o Struct)**: è il **progetto architettonico** (*blueprint*), un modello che definisce proprietà e comportamenti.
- **Oggetto (o Istanza)**: è la **casa concreta costruita** in memoria RAM a partire da quel progetto.

```mermaid
flowchart LR
    CLASSE["Classe: Giocatore\n(Progetto astratto)"]
    O1["Oggetto 1: g1\nNome: 'Mario'\nLivello: 5"]
    O2["Oggetto 2: g2\nNome: 'Luigi'\nLivello: 3"]

    CLASSE -->|"Istanziazione"| O1
    CLASSE -->|"Istanziazione"| O2
```

---

2. Le Struct (Strutture Dati)

Una `struct` è un tipo composto che raggruppa variabili di tipo differente, chiamate **campi** (o membri).

```
#include <iostream>
#include <string>
using namespace std;

struct Videogioco {
    string titolo;
    string genere;
    float valutazione;
    int annoUscita;
};

int main() {
    Videogioco g1; // Creazione istanza

    // Accesso ai campi con l'operatore punto (.)
    g1.titolo = "The Legend of Zelda";
    g1.genere = "Action-Adventure";
    g1.valutazione = 9.8f;
    g1.annoUscita = 2017;

    cout << "Gioco: " << g1.titolo << " (" << g1.annoUscita << ")" << endl
    return 0;
}
```

---

3. Dalle Struct alle Classi (OOP)

> ❓ **Domanda d'Esame: Qual è l'unica differenza tra una `struct` e una `class` in C++?**
>
> In C++ l'unica differenza sintattica è la **visibilità predefinita**:
>
> - Nelle `struct` : tutti i membri sono **`public`** di default.

> - Nelle `class`: tutti i membri sono **`private`** di default.

Per convenzione del settore:

- Si usa `struct` per meri contenitori di dati (*Plain Old Data*).
- Si usa `class` per la **programmazione ad oggetti (OOP)** basata su **incapsulamento**.

```
classDiagram
    class ContoBancario {
        -saldo: double
        -titolare: string
        +ContoBancario(titolare, saldoIniziale)
        +deposita(importo: double) void
        +preleva(importo: double) bool
        +getSaldo() double
    }
```

---

4. Incapsulamento: `public` vs `private`

L'**incapsulamento** protegge i dati interni dell'oggetto impedendo che vengano manipolati in modo errato dall'esterno.

- **`private`**: visibili solo alle funzioni interne della classe stessa.
- **`public`**: accessibili da chiunque (es. nel `main`).

> 🎯 **Regola d'Oro dell'Incapsulamento: I Getter e Setter**
>
> Metti sempre le variabili in `private` e fornisci metodi `public` per leggerle (`getSaldo`) e modificarle (`deposita`) con controlli di validità!

```
class ContoBancario {
private:
    double saldo; // Protetto: nessuno può impostare saldo = -99999 dall'es

public:
    // Metodo setter con validazione
    void deposita(double importo) {
        if (importo > 0) {
            saldo += importo;
            cout << "Depositati " << importo << " euro." << endl;
        } else {
            cout << "Importo non valido!" << endl;
        }
    }

    // Metodo getter (const indica che è di sola lettura!)
    double getSaldo() const {
        return saldo;
    }
};
```

---

5. Costruttori, Distruttori e il Puntatore `this`

Il Costruttore

Metodo speciale con lo **stesso nome della classe** e senza tipo di ritorno. Viene chiamato **automaticamente** alla nascita dell'oggetto:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Giocatore {
private:
    string nome;
    int puntiVita;
    int livello;

public:
    // 1. Costruttore di Default (senza parametri)
    Giocatore() : nome("Anonimo"), puntiVita(100), livello(1) {}

    // 2. Costruttore Parametrizzato con uso di this->
    Giocatore(string nome, int puntiVita, int livello) {
        // 'this' punta all'oggetto corrente: disambigua il campo dal para
        this->nome = nome;
        this->puntiVita = puntiVita;
        this->livello = livello;
    }

    // Metodo di stampa
    void stampaScheda() const {
        cout << "Guerriero: " << nome << " | PV: " << puntiVita << " | Liv
    }

    // Distruttore (eseguito alla distruzione dell'oggetto)
    ~Giocatore() {
        // Dealloca eventuale memoria dinamica nello Heap
    }
};

int main() {
    Giocatore g1;                       // Costruttore di default
    Giocatore g2("Artu", 150, 5);       // Costruttore parametrizzato

    g1.stampaScheda();
    g2.stampaScheda();
    return 0;
}
```

> 🖼️ **Placeholder Immagine: Diagramma delle Classi UML e Relazione con gli Oggetti**
>
> *Suggerimento per Obsidian: inserisci qui uno screenshot di un diagramma UML completo con classi, attributi e metodi.*
> `![[Pasted image uml_classi.png|550]]`

---

6. Puntatori a Oggetti e l'Operatore Freccia (`->`)

Quando lavoriamo con un puntatore a un oggetto o una struct, usiamo l'operatore `->` (freccia) invece della combinazione scomoda `(*ptr).` :

```
Giocatore *ptr = new Giocatore("Lancillotto", 120, 4);

// Invece di: (*ptr).stampaScheda();
ptr->stampaScheda(); // Molto più leggibile!

delete ptr; // Libera la memoria nello Heap
ptr = nullptr;
```

Tabella di Confronto: `struct` vs `class`

Proprietà	`struct`	`class`
Visibilità predefinita	`public`	`private`
Uso prevalente	Semplici aggregati di dati (*Plain Old Data*)	Programmazione OOP, logica di business protetta
Supporto a Metodi e Costruttori	Sì	Sì

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Puntatore**: variabile che memorizza l'**indirizzo di memoria RAM** di un'altra variabile.
> - **Operatore `&` (Indirizzo)**: ottiene la posizione fisica (`ptr = &x;`).
> - **Operatore `*` (Dereferenziazione)**: legge o scrive il **valore puntato** (`*ptr = 100;`).
> - **Inizializzazione**: usa sempre **`nullptr`** se non hai una variabile a cui puntare.
> - **Memoria Dinamica**: **`new`** alloca nello Heap a runtime; **`delete`** libera obbligatoriamente la RAM.

I **puntatori** consentono di interagire direttamente con la memoria RAM, offrendo massima velocità e controllo totale sull'hardware.

---

1. Variabili e Indirizzi di Memoria

Ogni volta che dichiariamo una variabile:

```
int numero = 42;
```

il sistema riserva nella RAM uno spazio di 4 byte. La cella possiede un **indirizzo fisico univoco** (in esadecimale, es. `0x7ffd5e8b41ac`).

```
flowchart LR
    subgraph RAM ["Memoria RAM"]
        Cella["Valore: 42<br>Indirizzo: 0x7ffd5e8b41ac<br>Nome: numero"]
    end

    OP_VAL["Accesso al valore: numero"] --> Cella
    OP_IND["Accesso all'indirizzo: &numero"] --> Cella
```

- Con `numero` leggiamo o scriviamo il **valore** (`42`).
- Con l'operatore **`&numero`** (*address-of*) otteniamo l'**indirizzo di memoria**.

---

2. Cos'è un Puntatore?

Un **puntatore** è una variabile speciale che **memorizza l'indirizzo di memoria di un'altra variabile**.

```mermaid
flowchart LR
    subgraph PUNTATORE ["Puntatore: int* ptr"]
        P_IND["Indirizzo: 0x2000"]
        P_VAL["Valore memorizzato: 0x1000"]
    end

    subgraph TARGET ["Variabile Bersaglio: int x"]
        T_IND["Indirizzo: 0x1000"]
        T_VAL["Valore: 50"]
    end

    P_VAL -->|"Punta all'indirizzo"| T_IND
```

---

3. I Due Operatori Fondamentali: `&` e `*`

Simbolo	Nome	Cosa fa	Esempio
`&`	Operatore Indirizzo (*address-of*)	Estrae la posizione in memoria	`int *p = &x;`
`*`	Operatore Dereferenziazione (*indirection*)	Accede al **contenuto** della cella puntata	`cout << *p;` oppure `*p = 99;`

```cpp
#include <iostream>
using namespace std;

int main() {
    int valore = 25;
    int *ptr = &valore; // 'ptr' punta alla cella di 'valore'

    cout << "Valore di valore:           " << valore << endl; // 25
    cout << "Indirizzo di valore (&):    " << &valore << endl; // 0x61ff08
    cout << "Valore memorizzato in ptr:  " << ptr << endl;    // 0x61ff08
    cout << "Valore puntato (*ptr):      " << *ptr << endl;   // 25

    // Modifica indiretta tramite puntatore:
    *ptr = 100;
    cout << "Nuovo valore di valore:     " << valore << endl; // 100!
    return 0;
}
```

> 🎯 **Regola d'Oro: Inizializzare sempre con `nullptr`**
>
> Se dichiari un puntatore senza assegnargli subito una variabile, inizializzalo a `nullptr`:
>
> ```cpp
> int *p = nullptr; // Evita che punti a una cella casuale e pericolosa
> ```

---

4. Puntatori vs Riferimenti (`&`)

> ❓ **Domanda d'Esame: Qual è la differenza tra un Puntatore e un Riferimento?**

Caratteristica	Puntatore (`int *p`)	Riferimento (`int &r`)
Può essere nullo?	**Sì** (`nullptr`)	**No**, deve riferirsi a una variabile reale
Riassegnabile?	**Sì**, può puntare ad altro	**No**, legato per sempre alla stessa cella
Sintassi di accesso	Richiede l'operatore `*p`	Trasparente (si usa come variabile comune)
Dimensione in memoria	Occupa 8 byte (a 64 bit)	Nessun overhead (è un semplice alias)

---

5. Puntatori e Array

In C++ il nome di un array **è già un puntatore costante al suo primo elemento**:

```
int arr[3] = {10, 20, 30};
int *p = arr; // Punta ad arr[0]

cout << *p << endl;       // 10 (arr[0])
cout << *(p + 1) << endl; // 20 (arr[1] - si sposta di 4 byte in avanti!)
cout << *(p + 2) << endl; // 30 (arr[2])
```

---

6. Memoria Dinamica: Stack vs Heap

```
flowchart TD
    subgraph RAM_ARCH ["Architettura della Memoria di un Programma"]
        STACK["STACK (Memoria Automatica)<br>• Veloce<br>• Dimensione fissa in compi
        GAP["Spazio libero"]
        HEAP["HEAP / Free Store (Memoria Dinamica)<br>• Controllata dallo sviluppato

        STACK --- GAP --- HEAP
    end
```

Gli Operatori `new` e `delete`

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Quanti elementi? ";
    cin >> n;

    // 1. Allocazione Dinamica nello Heap (new[])
    int *arr = new int[n];

    for (int i = 0; i < n; i++) arr[i] = (i + 1) * 10;

    // 2. Deallocazione Obbligatoria (delete[])
    delete[] arr;
    arr = nullptr; // Evita puntatori pendenti

    return 0;
}
```

---

7. I Tre Errori Mortali con i Puntatori

```mermaid
flowchart TD
    E1["1. Memory Leak: allochi con new ma dimentichi delete.<br>La RAM rimane occup
    E2["2. Dangling Pointer: fai delete su un'area ma continui a usare il puntatore.
    E3["3. Null Pointer Dereference: tenti di fare *ptr quando ptr == nullptr.<br>Cr

    E1 --- E2 --- E3
```

> 🚫 **Errore da Bocciatura: Il Memory Leak**
>
> A ogni `new` deve corrispondere **esattamente un** `delete`. Se allochi memoria in un ciclo senza liberarla, il computer esaurirà tutta la RAM disponibile fino al blocco di sistema!

> 🖼️ **Placeholder Immagine: Schema visivo di Memory Leak e Dangling Pointer**
>
> *Suggerimento per Obsidian: inserisci qui un disegno che illustra blocchi di memoria orfani nello Heap non più raggiungibili dal programma.*
> `![[Pasted image memory_leak.png|550]]`

> **In Sintesi (A Colpo d'Occhio)**
>
> - **Selezione**: `if-else` per condizioni booleane; `switch` per menu numerici o caratteri (ricorda il `break`!).
> - **Ciclo `for`**: ideale quando il **numero di iterazioni è noto** a priori (es. array da 0 a N).
> - **Ciclo `while`**: controllo in testa, esegue da **0 a N volte** (ottimo per lettura file).
> - **Ciclo `do-while`**: controllo in coda, esegue **almeno 1 volta** (la scelta d'oro per la validazione dell'input).
> - **Interruzioni**: `break` esce dal ciclo; `continue` salta al giro successivo.

Nei programmi reali dobbiamo consentire al computer di prendere decisioni o ripetere blocchi di istruzioni finché una condizione è verificata.

Queste capacità sono governate dalle **strutture di controllo**:

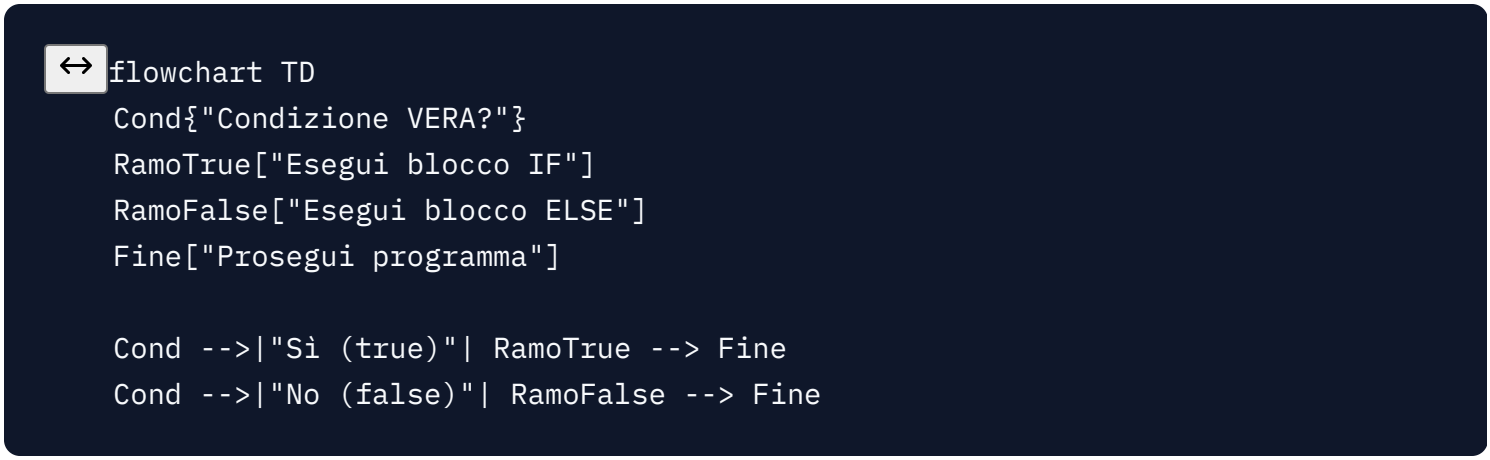
1. **Selezione (Condizionali)**: `if`, `else`, `switch`.
2. **Iterazione (Cicli)**: `while`, `do-while`, `for`.

---

1. Strutture di Selezione

Il costrutto `if` - `else if` - `else`

Biforca il flusso di esecuzione in base al valore booleano (`true` o `false`) dell'espressione.



```cpp
#include <iostream>
using namespace std;

int main() {
    int voto;
    cout << "Inserisci voto (1-10): ";
    cin >> voto;

    if (voto >= 8) {
        cout << "Livello: Ottimo" << endl;
    } else if (voto >= 6) {
        cout << "Livello: Sufficiente" << endl;
    } else {
        cout << "Livello: Insufficiente" << endl;
    }

    return 0;
}
```

Cortocircuito Logico (*Short-Circuit Evaluation*)

- In `cond1 && cond2`: se `cond1` è **falsa**, il computer **non valuta nemmeno** `cond2` (il risultato è per forza falso).
- In `cond1 || cond2`: se `cond1` è **vera**, il computer **non valuta** `cond2` (il risultato è per forza vero).

> 🎯 **Trucco del Mestiere: Sicurezza dei Puntatori**
>
> Grazie al cortocircuito puoi scrivere controlli salvavita come:
> `if (ptr != nullptr && *ptr > 0)`
> Se `ptr` è nullo, la dereferenziazione `*ptr` viene evitata proteggendo il programma da un crash!

Selezione Multipla: `switch`

Quando confrontiamo **una singola variabile intera o carattere** contro più valori costanti, `switch` è molto più pulito di una sfilza di `else if`.

```
flowchart TD
    Scelta{"Valore variabile"}
    C1["case 1: Azione A"]
    C2["case 2: Azione B"]
    DEF["default: Azione alternativa"]
    Uscita["Uscita dallo switch"]

    Scelta -->|"== 1"| C1 -->|"break"| Uscita
    Scelta -->|"== 2"| C2 -->|"break"| Uscita
    Scelta -->|"Altro"| DEF -->|"break"| Uscita
```

```
char scelta;
cout << "Vuoi continuare? (s/n): ";
cin >> scelta;

switch (scelta) {
    case 's':
    case 'S':
        cout << "Continuo l'elaborazione..." << endl;
        break; // Esce dallo switch!
    case 'n':
    case 'N':
        cout << "Operazione annullata." << endl;
        break;
    default:
        cout << "Scelta non riconosciuta!" << endl;
        break;
}
```

> **🚫 Errore da Matita Rossa: Dimenticare il `break` (Fall-Through)**
>
> Se ometti l'istruzione `break`, il programma continuerà a eseguire indistintamente anche i `case` sottostanti anche se la condizione non corrisponde!

2. Strutture di Iterazione (I Cicli)

```mermaid
flowchart LR
    subgraph WHILE_LOOP ["while (Pre-condizionale)"]
        W1{"Condizione?"} -->|"true"| W2["Corpo del ciclo"] --> W1
        W1 -->|"false"| W3["Fine (0 o più volte)"]
    end

    subgraph DOWHILE_LOOP ["do-while (Post-condizionale)"]
        D1["Corpo del ciclo"] --> D2{"Condizione?"}
        D2 -->|"true"| D1
        D2 -->|"false"| D3["Fine (almeno 1 volta)"]
    end
```

> ❓ **Domanda d'Esame: Differenza tra `while` e `do-while`**
>
> - `while` : la condizione viene controllata **prima**. Se è falsa subito, il ciclo esegue **0 iterazioni** .
> - `do-while` : la condizione viene controllata **dopo**. Il corpo viene eseguito **almeno 1 volta garantita** .

Lo Schema Fisso della Validazione Input con `do-while`

> 🎯 **Il Pattern per la Verifica dei Dati Utente**

```
int voto;
do {
    cout << "Inserisci un voto valido (1-10): ";
    cin >> voto;
} while (voto < 1 || voto > 10); // Ripete se il dato è SBAGLIATO!
```

🖼️ **Placeholder Immagine: Diagramma di flusso della validazione di un input**

Suggerimento per Obsidian: inserisci qui un diagramma di flusso flowchart della validazione utente con do-while.

`![[Pasted image validazione_flowchart.png|500]]`

Il Ciclo `for` (A Conteggio)

↔
```
flowchart TD
    Init["1. Inizializzazione: int i = 0"] --> Test{"2. Test Condizione: i < N"}
    Test -->|"Sì (true)"| Corpo["3. Esecuzione Corpo"]
    Corpo --> Incr["4. Incremento: i++"] --> Test
    Test -->|"No (false)"| Esci["5. Uscita dal ciclo"]
```

```
// Stampa i numeri da 1 a 5
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
cout << endl;
```

---

3. Istruzioni di Salto: `break` e `continue`

- `break` : termina ed esce all'istante dal ciclo più interno.

- `continue` : **salta il resto del giro corrente** e passa subito all'iterazione successiva.

```
for (int i = 1; i <= 6; i++) {
    if (i == 3) continue; // Salta il numero 3!
    if (i == 5) break;    // Blocca il ciclo prima del 5!
    cout << i << " ";
}
// Stampa a schermo: 1 2 4
```

Tabella di Scelta Rapida

Situazione	Struttura Consigliata	Esempio Tipico
Numero di iterazioni noto a priori	**`for`**	Scorrere un array di 50 elementi
Controllo preventivo della condizione	**`while`**	Lettura da file riga per riga fino a EOF
Esecuzione obbligatoria almeno una volta	**`do-while`**	Validazione da tastiera, menu ripetuto
Scelta multipla tra valori fissi	**`switch`**	Menu di selezione (1, 2, 3...)